Лабораторная работа №4. Docker Compose

Содержание

Содержание

1 Цели лабораторной работы **3**

2 Задачи к лабораторной работе **3**

3 Методические материалы **4**

3.1 Что такое Docker Compose? 4

3.2 Хранение данных 4

3.3 Сети 6

3.4 Описание серисов через docker-compose.yml 7

3.5 Основные команды Docker Compose CLI 13

4 Ход работы **15**

5 Контрольные вопросы **15**

1 Цели лабораторной работы

- Познакомиться с технологией Docker Compose;
- Изучить основные команды CLI;
- Изучить формат описания сервисов с помощью *docker-compose.yml*.

2 Задачи к лабораторной работе

1. Изучить методические материалы к лабораторной работе;
2. Описать и развернуть три сервиса с помощью *docker-compose.yml*.

3 Методические материалы

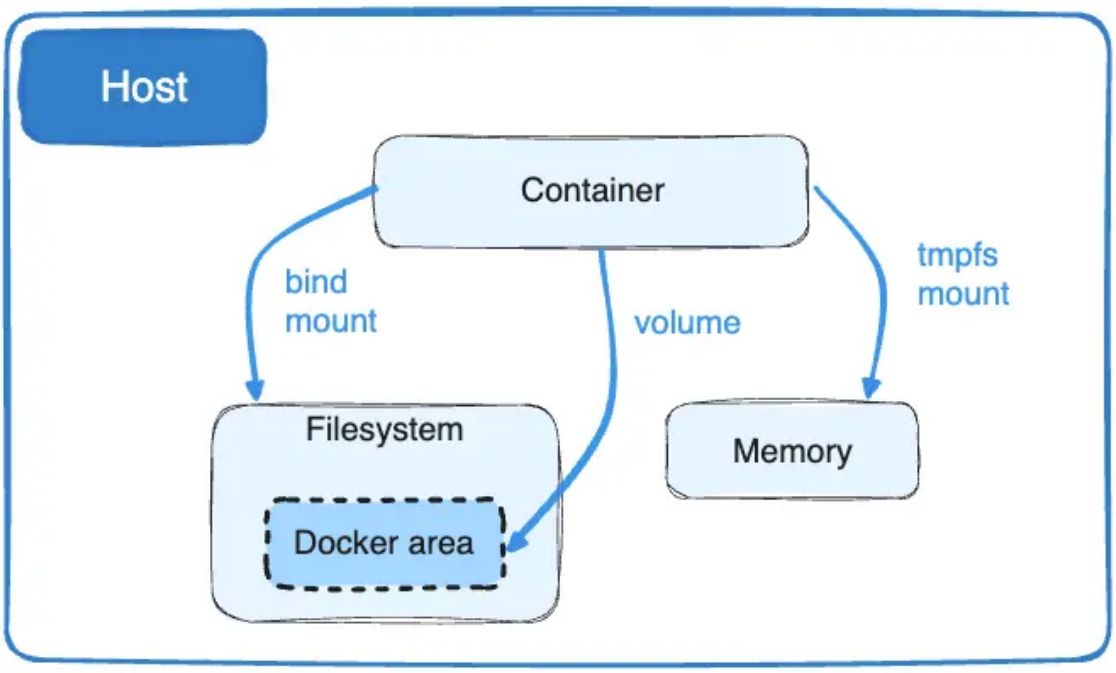
3.1 Что такое Docker Compose?

Docker Compose - это инструмент для определения и запуска многоконтейнерных приложений. Compose упрощает управление всем стеком приложений, позволяя легко манипулировать сервисами, сетями и томами в едином и понятном конфигурационном файле YAML. Затем, с помощью одной команды, создать и запустить все сервисы из файла конфигурации.

Прежде чем приступить к описанию сервисов с помощью файла конфигурации, необходимо разобраться ещё с двумя концепциями Docker: хранение данных и сети.

3.2 Хранение данных

Контейнеры - это эфемерные сущности, то есть непостояны. Они могут запускаться и останавливаться по множеству раз из-за разных причин. При удалении контейнера также удаляется и всё его содержимое, однако зачастую требуется сохранять данные между перезапусками контейнеров (к примеру, файлы баз данных). Docker предоставляет механизм хранения данных дольше жизни контейнера: тома (volumes) и связывающие монтирования (bind mounts).



Тома (Volumes). Тома являются предпочтительным механизмом для сохранения данных, создаваемых и используемых контейнерами Docker. В то время как связывающее монтирование зависит от структуры каталогов и ОС хост-машины, тома полностью управляются Docker. У томов есть несколько преимуществ по сравнению с bind mounts:

- Резервное копирование или перенос томов проще, чем при связывающем монтировании;
- Возможность управлять томами с помощью команд Docker CLI или Docker API;
- Тома можно безопаснее распределять между несколькими контейнерами;
- Тома более безопасны, так как имеют бóльшую степень изолированности;
- Драйверы томов позволяют хранить тома на удаленных узлах или у облачных провайдеров, шифровать содержимое томов и добавлять другие функции.

Тома отлично подходят для хранения данных, которые создаются во время работы контейнера: файлы БД, файлы логгирования и прочие. Однако они плохо справляются с задачами конфигурирования приложений внутри контейнера.

Связывающие монтирования (Bind mounts). Bind mounts имеют ограниченную функциональность по сравнению с томами. При использовании связывающего монтирования файл или каталог на хост-машине монтируется в контейнер. Ссылка на файл или каталог осуществляется по его абсолютному пути на хост-машине. В отличие от этого, при использовании тома создается новый каталог в каталоге хранения Docker на хост-машине, и Docker управляет содержимым этого каталога.

Файл или каталог не обязательно должен уже существовать на хосте Docker. Он создается по требованию, если еще не существует. Связывающие монтирования очень производительны, но они зависят от наличия в файловой системе хост-машины определенной структуры каталогов. Также вы не можете использовать команды Docker CLI для прямого управления связывающими монтированиями.

Bind mounts часто используется для передачи файлов конфигурации в контейнер. Так как файлы конфигурации необходимы только для чтения содержимого из них и в целях защиты от записи в файл/каталог, смонтированного напрямую с хоста в контейнер, Docker предоставляет возможность создания read-only bind mounts.

3.3 Сети

Сетевое взаимодействие контейнеров - это возможность контейнеров подключаться и взаимодействовать друг с другом или с рабочими нагрузками, не относящимися к Docker.

По умолчанию у контейнеров включена сетевая связь, и они могут устанавливать исходящие соединения. Контейнер не имеет информации о том, к какой сети он подключен, а также о том, являются ли его коллеги рабочими нагрузками Docker или нет. Контейнер видит только сетевой интерфейс с IP-адресом, шлюз, таблицу маршрутизации, службы DNS и другие сетевые детали. Это если контейнер не использует сетевой драйвер none.

Docker позволяет создавать пользовательские сети и подключать несколько контейнеров к ним. После подключения к пользовательской сети контейнеры могут взаимодействовать друг с другом, используя IP-адреса или имена контейнеров.

Для управления сетями Docker существует команда `docker network` с подкомандами `create`, `rm`, `ls`, `inspect`, `prune`, `connect` и `disconnect`.

Чтобы создать новую пользовательскую сеть, используйте:

```
$ docker network create <network_name>
```

Драйверы. Сетевые драйверы - это подключаемые интерфейсы, которые обеспечивают фактическую реализацию сети для контейнеров Docker. В основном они необходимы для того, чтобы определить способ взаимодействия контейнеров с внешним миром.

Драйвер	Описание
bridge	Драйвер по умолчанию
host	Убирает сетевую изоляцию между контейнером и хостом Docker
none	Полностью изолирует контейнер от хоста и других контейнеров
overlay	Оверлейные сети соединяют несколько демонов Docker вместе
ipvlan	Сети IPvlan обеспечивают полный контроль над адресацией как IPv4, так и IPv6
macvlan	Назначение MAC-адреса контейнеру

При желании можете более подробно изучить информацию про сетевые драйверы.

3.4 Описание серисов через docker-compose.yml

Для принятия решения о запускаемых сервисах Docker Compose считывает содержимое файла docker-compose.yml. Далее будет приведена основная информация, необходимая для запуска контейнеров через Docker Compose.

```
services:
    web:                            # название сервиса
        image: nginx:latest         # используемый образ
        ports:                      # список пробрасываемых портов
            - "8080:80"             # элемент списка

        volumes:                    # список постоянных хранилищ
            - ./nginx.conf:/etc/nginx/nginx.conf

    db:
        image: postgres:13
        restart: always             # политика перезагрузки контейнера
        environment:                # переменные окружения
            POSTGRES_USER: example
            POSTGRES_DB: exampledb

        volumes:
            - postgres_data:/var/lib/postgresql/data

        networks:                   # список сетей
```

```
            - net1

networks:
    net1:                               # имя сети
        external: true                  # указание, что сеть создана извне

volumes:
    postgres_data:                      # имя тома
```

Пример docker-compose.yml файла

Элементы верхнего уровня сервисов. **Сервис** - это абстрактное определение вычислительного ресурса в приложении, который можно масштабировать или заменять независимо от других компонентов. Сервисы поддерживаются набором контейнеров, запускаемых платформой в соответствии с требованиями к репликации и ограничениями на размещение. Поскольку сервисы поддерживаются контейнерами, они определяются образом Docker и набором аргументов времени выполнения. Все контейнеры внутри сервиса создаются одинаково с этими аргументами.

В файле Compose должен быть объявлен элемент верхнего уровня `services`, содержащий ключи, которыми являются строковые представления имен сервисов, а значениями - определения сервисов. Определение сервиса содержит конфигурацию, которая применяется к каждому контейнеру сервиса.

В документации вы можете найти весь перечень возможных значений для элементов верхнего уровня сервисов. Мы рассмотрим основные:

- `image: <image_name>` - указывает имя образа. Если на машине нет образа с таким названием, то Docker попытается найти его в Docker Registry.

  ```
  services:
      my-service:
          image: my-image
  ```

- `build: <path>` - указывает путь до директории, содержащей Dockerfile для этого сервиса относительно docker-compose.yml файла.

  ```
  services:
      my-service:
          build: .
  ```

- `ports` - список прокидываемых с хоста в контейнер портов.

```
services:
    my-service:
        ports:
            - "80:80"
            - "443:443"
```

Слева от двоеточия указывается порт на хосте, справа - в контейнере.

- `environment` - список переменных окружения. Может быть задан как `<key>:<value>` пара или список `<key>=<value>` значений:

```
services:
    my-service:
        environment:
            POSTGRES_USER: example
            POSTGRES_DB: exampledb

# или

services:
    my-service:
        environment:
            - POSTGRES_USER=example
            - POSTGRES_DB=exampledb
```

Обратите внимание: во втором случае пробелы не ставятся вокруг символа присвоения.

- `volumes` - список постоянных хранилищ. В качестве значений могут быть указаны как тома, так и связывающие монтирования.

```
services:
    my-service:
        volumes:
            - ./nginx.conf:/etc/nginx/nginx.conf
            - postgres_data:/var/lib/postgresql/data
```

В правой от двоеточия части указывается путь монтирования внутри контейнера.

Для связывающего монтирования в левой части указывается путь до ресурса (директории или файла) на хосте. Однако, если вы хотите смонтировать внутрь контейнера директорию «secret_data», которая находится на одном уровне с docker-compose.yml файлом, то используйте явное указание относительного пути: «./secret_data», иначе это будет принято за именнованный том и при его отсутствии запуск сервиса завершится неудачей.

Для тома указывается его имя. Соотвтествующие имена томов, указанные в сервисах, должны быть определены в элементах верхнего уровня томов (об этом далее).

- `networks` - список сетей, в которые будет добавлен сервис.

```
services:
    my-service:
        networks:
            - net1
            - net2
```

Соотвтествующие имена сетей, указанные в сервисах, должны быть определены в элементах верхнего уровня сетей (об этом далее).

- `restart` - определяет политику, которую платформа применяет при завершении работы сервиса. Возможные значения:
  - `no` - Политика перезапуска по умолчанию. Она не перезапускает контейнер ни при каких обстоятельствах;
  - `always` - Политика всегда перезапускает контейнер до его удаления;
  - `on-failure[:max-retries]` - Политика перезапускает контейнер, если код выхода указывает на ошибку. Опционально можно ограничить количество попыток перезапуска;
  - `unless-stopped` - Политика перезапускает контейнер независимо от кода выхода, но прекращает перезапуск, когда служба остановлена или удалена.

Элементы верхнего уровня томов. **Тома** - это постоянные хранилища данных, реализованные в контейнерном движке. Compose предлагает нейтральный способ

для сервисов монтировать тома, а также параметры конфигурации для распределения их по инфраструктуре. Объявление томов верхнего уровня позволяет настраивать именованные тома, которые можно повторно использовать в нескольких службах.

Чтобы использовать том в нескольких службах, необходимо явно предоставить доступ каждой службе с помощью атрибута volumes в элементе верхнего уровня services. Атрибут volumes имеет дополнительный синтаксис, который обеспечивает более детальный контроль.

В самом простом случае это будет выглядеть так:

```
volumes:
    volume_name1:
    volume_name2:
```

Основные атрибуты:

- `name` - задаёт пользовательское имя тома. Поле name можно использовать для ссылок на тома, содержащие специальные символы.

  ```
  volumes:
      app-db:
          name: "app-data"
  ```

- `driver` - указывает, какой драйвер тома должен быть использован. Если драйвер недоступен, Compose возвращает ошибку и не развертывает приложение.

  ```
  volumes:
      app-db:
          driver: local
  ```

- `external`. Если установлено в `true`:
  - указывает, что этот том уже существует на платформе и его жизненный цикл управляется вне приложения. В этом случае Compose не создает том и возвращает ошибку, если том не существует;
  - все остальные атрибуты, кроме имени, не имеют значения. Если Compose обнаружит любой другой атрибут, он отвергнет файл как недействительный.

```
volumes:
    app-db:
        external: true
```

О том, какие ещё атрибуты есть у элементов, можно посмотреть в документации.

Элементы верхнего уровня сетей. Сети позволяют сервисам взаимодействовать друг с другом. По умолчанию Compose создает одну сеть для вашего приложения. Каждый контейнер для сервиса присоединяется к сети по умолчанию и доступен для других контейнеров в этой сети, а также обнаруживается по имени сервиса. Элемент сети верхнего уровня позволяет настраивать именованные сети, которые можно повторно использовать в нескольких службах.

Чтобы использовать сеть в нескольких службах, необходимо явно предоставить доступ каждой службе с помощью атрибута networks в элементе верхнего уровня services. Элемент верхнего уровня networks имеет дополнительный синтаксис, который обеспечивает более детальный контроль.

```
networks:
    net1:
    net2:
```

Весь перечень атрибутов можно посмотреть в документации. Основные атрибуты:

- `name` - задает пользовательское имя сети. Поле имени можно использовать для ссылки на сети, содержащие специальные символы.

  ```
  networks:
      my-net:
          name: other-net-name
  ```

- `driver` - указывает, какой драйвер должен использоваться для данной сети. Compose возвращает ошибку, если драйвер недоступен на данной платформе.

  ```
  networks:
      my-net:
          driver: host
  ```

- `external`. Если установлено в `true`:

– указывает, что жизненный цикл этой сети поддерживается вне приложения. Compose не пытается создавать такие сети и возвращает ошибку, если такой сети не существует;

– все остальные атрибуты, кроме имени, не имеют значения. Если Compose обнаружит любой другой атрибут, он отвергнет файл как недействительный.

```
networks:
    my-net:
        external: true
```

3.5 Основные команды Docker Compose CLI

Основные команды, которыми вы будете пользоваться - это `docker compose up` и `docker compose down`.

docker compose up. Сборка, (пере)создание, запуск и прикрепление контейнеров для сервиса. Если они еще не запущены, эта команда также запускает все связанные сервисы.

```
$ docker compose up [OPTIONS] [SERVICE...]
```

Основные флаги:

- `-d, --detach` - запуск контейнеров в фоновом режиме;
- `--build` - создание образов перед запуском контейнеров.

docker compose down. Остановка и удаление контейнеров, сетей, томов и образов, созданных при запуске. По умолчанию удаляются только:

- контейнеры для сервисов, определенных в файле Compose;
- сети, определенные в разделе сетей файла Compose;
- сеть по умолчанию, если таковая используется.

Сети и тома, определенные как внешние, никогда не удаляются.

```
$ docker compose down [OPTIONS] [SERVICES]
```

Основные флаги:

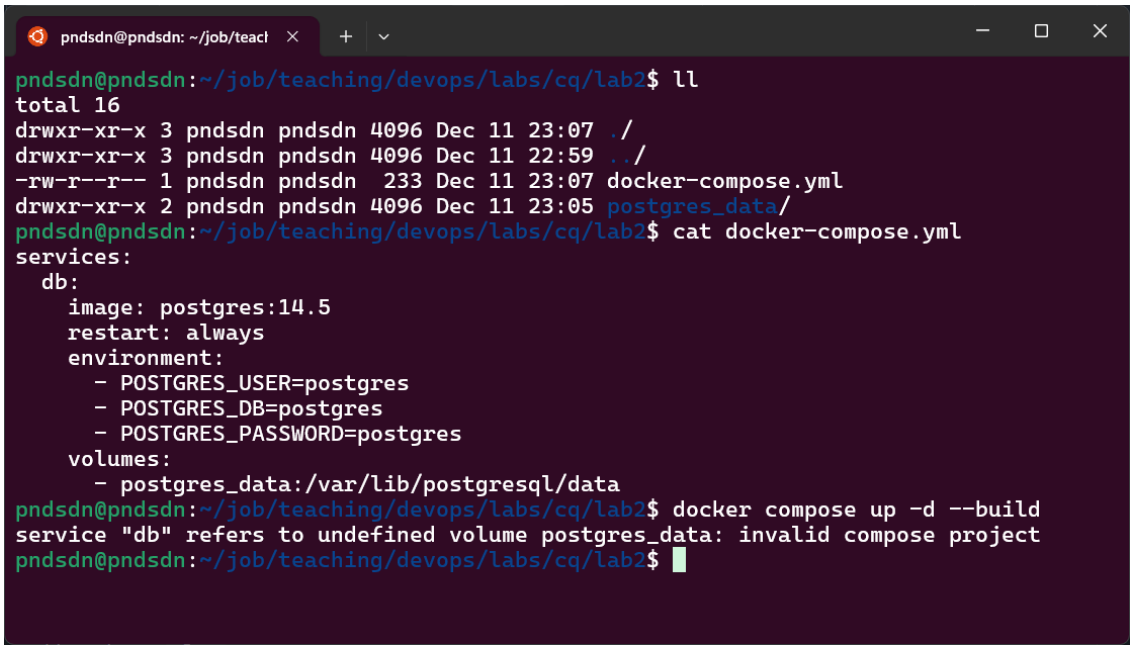
- `-v, --volumes` - удаление именованных томов, объявленных в разделе «тома» файла Compose, и анонимных томов, прикрепленных к контейнерам;
- `--remove-orphans` - удаление контейнеров для сервисов, не определенных в файле Compose;
- `--rmi` - удаление образов, используемых сервисами. Удаляет только те образы, у которых нет пользовательского тега.

4 Ход работы

Склонируйте репозиторий проекта и следуйте инструкциям из README.md.

5 Контрольные вопросы

- Что такое Docker Compose и для чего он используется?
- На рисунке ниже продемонстрирован docker-compose.yml, в котором описан один сервис db. При попытке смонтировать каталог `postgres_data` внутрь контейнера для хранения записей базы данных происходит ошибка. Почему возникает ошибка и как её исправить?



Демонстрация примера